FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433 SOFTWARE ARCHITECTURE

PROJECT TITLE:

CLOUD NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

GROUP 4

Name	Matric No.
EASHENRAJ A/L NAGARAJAN	S76773
MOHAMMAD ADAM BASHEER B. MOHD AZRUL	S75710
AMMAR HAZIQ B. SAIFUL BAHRI	S75772

PREPARED FOR:

DR MOHAMAD NOR HASSAN

Table of Content

1. Architecture Design

1.1 System Overview

The Learning Management System (LMS) is a platform designed to manage educational activities. These activities consist of course creation, enrollment, content delivery, assignment submission, and grading. This system will follow the Layered Architecture Pattern, where it will ensure separation of concerns between the presentation layer, business logic layer, data access layer, and database layer.

This architectural pattern can improve the system's maintainability, scalability, and security by isolating each layer's responsibilities. The system will use RESTful API endpoints to ensure there is structured communication between the frontend and backend. The system will expose well-defined RESTful API endpoints, which can be tested and validated using API testing tools such as Postman.

1.2 Selected Architecture Style

The architecture style that will be used in this project is the Layered Architecture Pattern. This design consists of different logical layers for an application, where each layer has a defined responsibility and communicates only with its adjacent layers. This approach of having different layers improves maintainability because components can be modified more easily, and scalability by allowing the system to scale different parts separately. These distinct layers are responsible for specific sets of functionalities. This architecture minimizes dependencies between layers, enabling greater flexibility and easier modification of individual components. For this LMS, we are using a closed layered approach, which means a layer cannot bypass another layer to access the lower layer.

The Presentation Layer is responsible for user interface design and user experience. This layer handles visual displays and user input, then communicates with the Business Layer to display processed results.

The Business Layer is the layer where information received from the previous layer is processed. This layer handles complex operations and core business rules. Having a separate layer allows modification to business rules without changing the user interface or database.

The Data Access Layer is where the data interactions happen. This layer ensures communication between the system and data storage. This layer handles CRUD operations such as create, read, update, and delete records. This layer also acts as a gatekeeper to ensure only authorized requests reach the data.

Finally, the Database Layer consists of actual database servers, which can be either relational database management systems or NoSQL databases. This layer is designed to be independent, so we can utilize a cloud database to improve reliability.

This architecture pattern allows our project to achieve Separation of Concerns, where each layer can be modified without making changes to other layers, and also allows each layer to be tested independently using test data, which helps to reduce the risk of bugs in the system.

1.3 Problem Description

Traditional Learning Management Systems (LMS) often suffer from a monolithic and tightly coupled design where the layers, such as user interface, business logic, and database, cause problems.

In traditional learning management system (LMS) designs, the user interface, business logic, and database queries are usually associated with each other. This makes system maintenance more complex. As a result, updating one feature may unintentionally break other features. This problem leads to long downtimes and high risks during system updates. Peak academic periods, such as course registration, cause traffic problems in the LMS due to the students attempting to access the database at the same time, which can cause the system to crash because the platform lacks the scalability and performance to handle a high volume load.

Other than that, the absence of a Data Access Layer (DAL) in a student learning management system (LMS) may cause tight coupling between the user interface, business logic, and database. Accessing features such as course materials and submitting assignments may directly interact with the database, which may cause corruption in the database. This makes the system less reliable and harder to scale.

Lastly, the traditional LMS design may create a scaling problem because the system is not modularized. Peak academic periods, such as course registration, cause traffic problems in the LMS due to the students attempting to access the database at the same time. This causes the system to crash because the platform lacks the scalability and performance to handle a high volume load.

1.4 System Objectives

The main goal of this project is to develop a scalable and maintainable Learning Management System (LMS) by applying modern architectural principles and design. The objectives are:

1. **To separate system components into distinct layers**
    - By utilizing the Layered Architecture Pattern, it can ensure that the modifications done to the user interface will not interrupt the other important parts, such as database integrity.
2. **To ensure secure data handling**
    - Establish a dedicated Data Access Layer (DAL) using Data Access Objects (DAO), protecting the system from data corruption and unauthorized access.
3. **To support scalable performance during peak usage periods**
    - Design the system structure to scale resources in response to high user demand. This specifically targets the performance drops experienced during course peak hours, such as registration or exam tests, ensuring the system remains stable despite high volumes of concurrent traffic.
4. **To reduce maintenance complexity**
    - Reduce system downtime by creating a structure where individual components can be updated, tested, and deployed independently without affecting the other components.
5. **To provide high performance**
    - Leverage cloud environments to ensure the LMS is accessible 24/7, providing a responsive experience for both students and lecturers to access the materials.

1.5 Functional Requirements

To effectively demonstrate a layered, cloud-native architecture, the system's functional requirements are categorized into several modules. This ensures the separation of concerns across the presentation, business logic, and data access layers.

1. User Identity & Access Management Module
    a. User Registration: The system shall allow new users to create an account by providing basic details such as name, email, password, and roles.
    b. Authentication: The system shall securely authenticate users upon login and generate a session token to manage access across different layers of the application.
    c. Role-Based Access Control: The system shall enforce role-specific permissions, for example, by preventing a Student from accessing a course-creation API endpoint.
    d. Profile Management: Users shall be able to view and update their personal profile information and change their passwords.

2. Course Management Module
    a. Course Creation: The system shall allow instructors to create a new course by defining a title, description, syllabus, and enrollment capacity.
    b. Course Modification: Instructors shall be able to update or delete courses they have created.
    c. Course Catalog: The system shall display a catalog of all available courses, allowing Students to search or filter by course name or instructor.
    d. Enrollment Processing: The system shall allow Students to enroll in available courses and automatically update their personalized dashboard to reflect active enrollments.

3. Content Delivery and Management Module
    a. Material Upload: The system shall allow Instructors to upload educational materials such as PDF documents, slide decks, or links to external resources to specific courses.
    b. Content Organization: Instructors shall be able to organize uploaded materials into modules or weeks for structured learning.
    c. Material Access**:** The system shall allow enrolled Students to view, access, and download course materials from the frontend interface.

4. Assignment and Assessment Module
    a. Assignment Creation: Instructors shall be able to create assignments, specifying the title, instructions, maximum score, and a strict submission deadline.
    b. File Submission: The system shall allow Students to upload file submissions for active assignments before the specified deadline.
    c. Submission Tracking: The system shall track and display the submission's status, such as “Submitted”, “Late”, “Pending”, or “Overdue” on the Student's dashboard.
    d. Grading and Feedback: Instructors shall be able to view student submissions, assign a numerical grade, and provide text-based feedback, which the system will make visible to the respective Student.

5. System & Architectural Requirements
    a. API Communication: The system's frontend layer shall communicate with the backend business logic exclusively through well-defined RESTful API endpoints.
    b. Data Persistence: The system shall accurately store and retrieve all user, course, and assignment data using a designated database.

1.6 Non-Functional Requirements

1. Performance

Performance ensures the system remains responsive under pressure, prioritizing consistency over just speed.

a. The system must support a minimum of 100 concurrent users without significant degradation in response time.
b. RESTful API endpoints should return responses within 2 seconds under normal operating conditions.

2. Scalability

Scalability is how the system can handle growth without changing its overall design.

a. The system shall support horizontal scaling, meaning additional server instances can be added to distribute load during periods of high traffic.
b. The Layered Architecture enables each layer, such as the Business Logic Layer or Data Access Layer, to be scaled independently based on demand.

3. Security

This non-functional requirement prioritizes data integrity and privacy, which ensures confidentiality.

a. Role-Based Access Control shall be enforced to prevent unauthorized access to restricted API endpoints and system features.
b. User passwords shall be stored using a strong hashing algorithm such as bcrypt, ensuring plain-text passwords are never stored or transmitted.

4. Availability

Availability measures the time the system is fully operational and accessible, also known as uptime. This ensures business continuity, reliability, and high performance.

a. The system shall target a minimum uptime of 99% following deployment.
b. System updates and bug fixes shall be deployable at the individual layer level, minimizing downtime by avoiding full system restarts.

5. Maintainability

Maintainability refers to the ease with which the system can be understood, updated, and extended over time.

a. The Layered Architecture separates the system into independent layers such as, Presentation, Business Logic, and Data Access which allow each to be modified or tested without impacting the others.
b. Loose coupling between components ensures that changes to one module do not require changes across unrelated parts of the system.

1.7 Service/Component Identification

Presentation Layer

This layer is responsible for handling every user interaction by displaying the system user interface. It is developed using HTML, CSS and JavaScript which will provide user interfaces such as course pages and assignments. Deployment happens on Netlify which will provide a fast content delivery, high availability and scalability for the system.

Business Logic Layer

This layer handles the processing of application logic. It is implemented using Spring Boot framework and RESTful API as a communication with the frontend of the system. These layers will handle the core system functionalities such as

- User authentication
- Course management
- Assignment handling

Deployed on Render which will provide a cloud hosting for backend services, automatic scaling and reliable uptime.

Data Access Layer

This layer manages the communication that happens between the backend and the database. It implements the DAO pattern to separate the database logic from the business logic. It handles CRUD operations such as UserDAO, CourseDAO, AssignmentDAO and ProgressDAO which will ensure the consistency and security of the system data.

Database Layer

This layer is responsible for data storage of the LMS system. Aiven MySQL is used as a database service which stores structured data such as:

- User Account
- Content
- Courses
- Assignments

Provides automated backups, scalability for growing data and secure database access.

1.8 System Decomposition

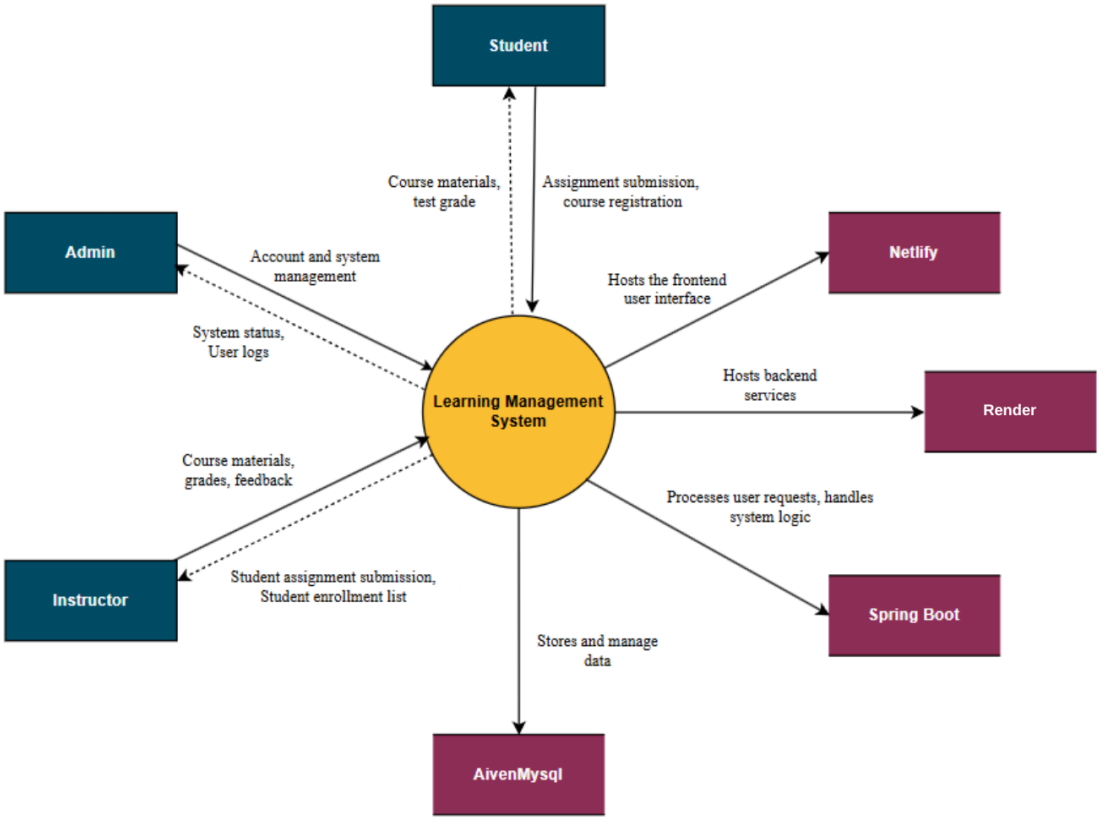
Component	Responsibility	Exclusions
Web Client (React SPA/Vite)	• Renders the user interface • Get user input • Sends HTTP requests to the API Gateway • Displays responses	• Does not process business logic • Does not access the database directly
API Gateway	Routes incoming HTTPS REST requests to the correct backend service	• Does not perform business logic • Does not store data
Auth Service	Handles user login, registration, JWT token generation, and validation	Does not manage course, assignment, or content data
Profile Service	Manages the viewing and updating of user personal information.	Does not handle authentication
Course Service	Handles course creation, modification, deletion, search, and enrollment processing	Does not handle assignment submission or file content uploads
Assignment Service	Handles assignment creation, file submission, deadline checking, submission status tracking (Submitted, Late, Pending, Overdue), grading, and feedback.	Does not manage course content or user profiles

Content Service	Manages the upload, and delivery of course materials	Does not handle authentication or assignment deadlines
User DAO	Executes CRUD operations on the Users table for user accounts, profiles, and roles	● Does not contain business logic ● Does not access any other table
Course DAO	Executes CRUD operations on the Courses table for course details and schedules	● Does not contain business logic ● Does not access any other table
Assignment DAO	Executes CRUD operations on the Assignments table for assignments, submissions, grades, and feedback	● Does not contain business logic ● Does not access any other table
Content DAO	Executes CRUD operations on the Contents table for content metadata and file references	● Does not contain business logic ● Does not access any other table
Aiven MySQL	Persists all structured data across four tables: Users, Courses, Assignments, Contents	● Does not process application logic ● Does not communicate directly with the frontend

1.9 Architecture Diagrams

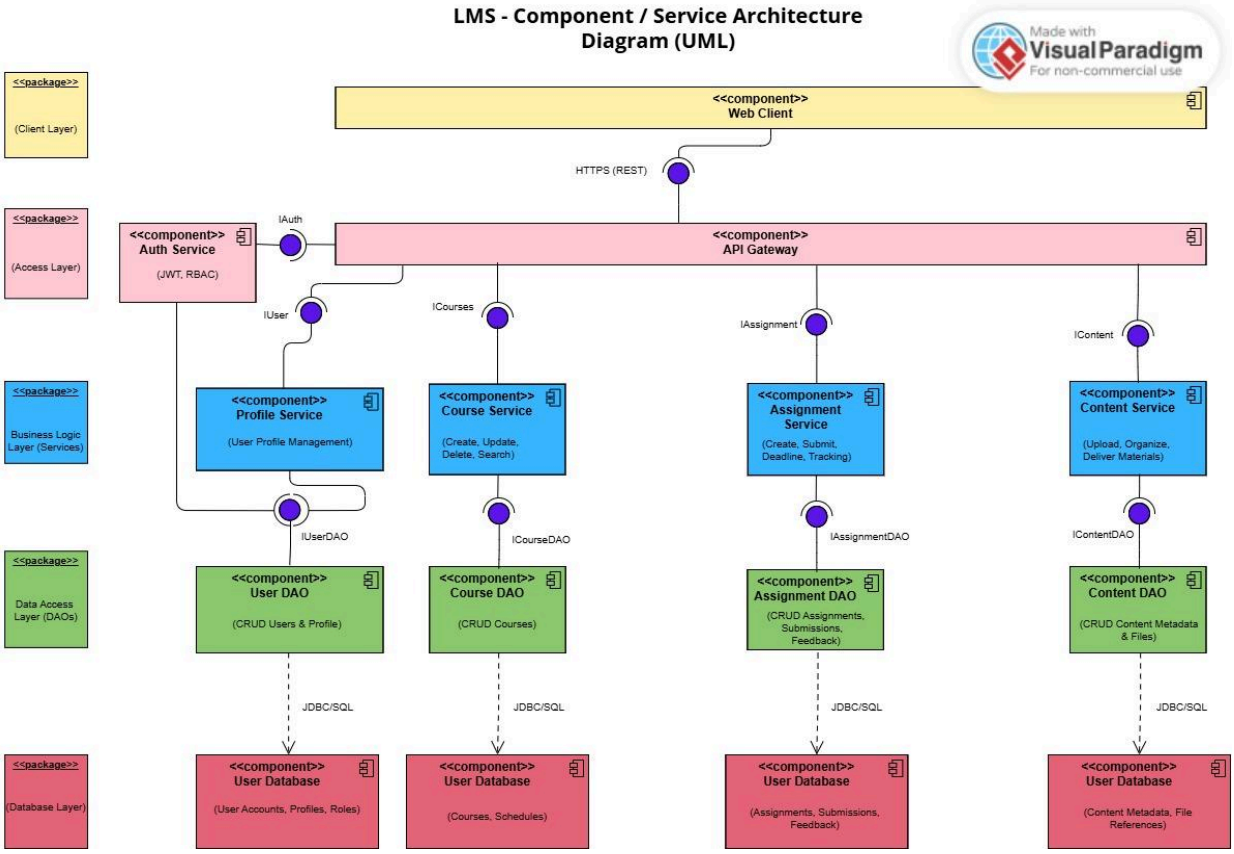
This section details the blueprint of the Learning Management System, providing a comprehensive overview of how different components, data flows, and infrastructure elements work together. By moving from high-level system boundaries to specific runtime interactions and physical deployment, these diagrams ensure a clear understanding of the application's scalability and modular design.

1.9.1 System Context Diagram



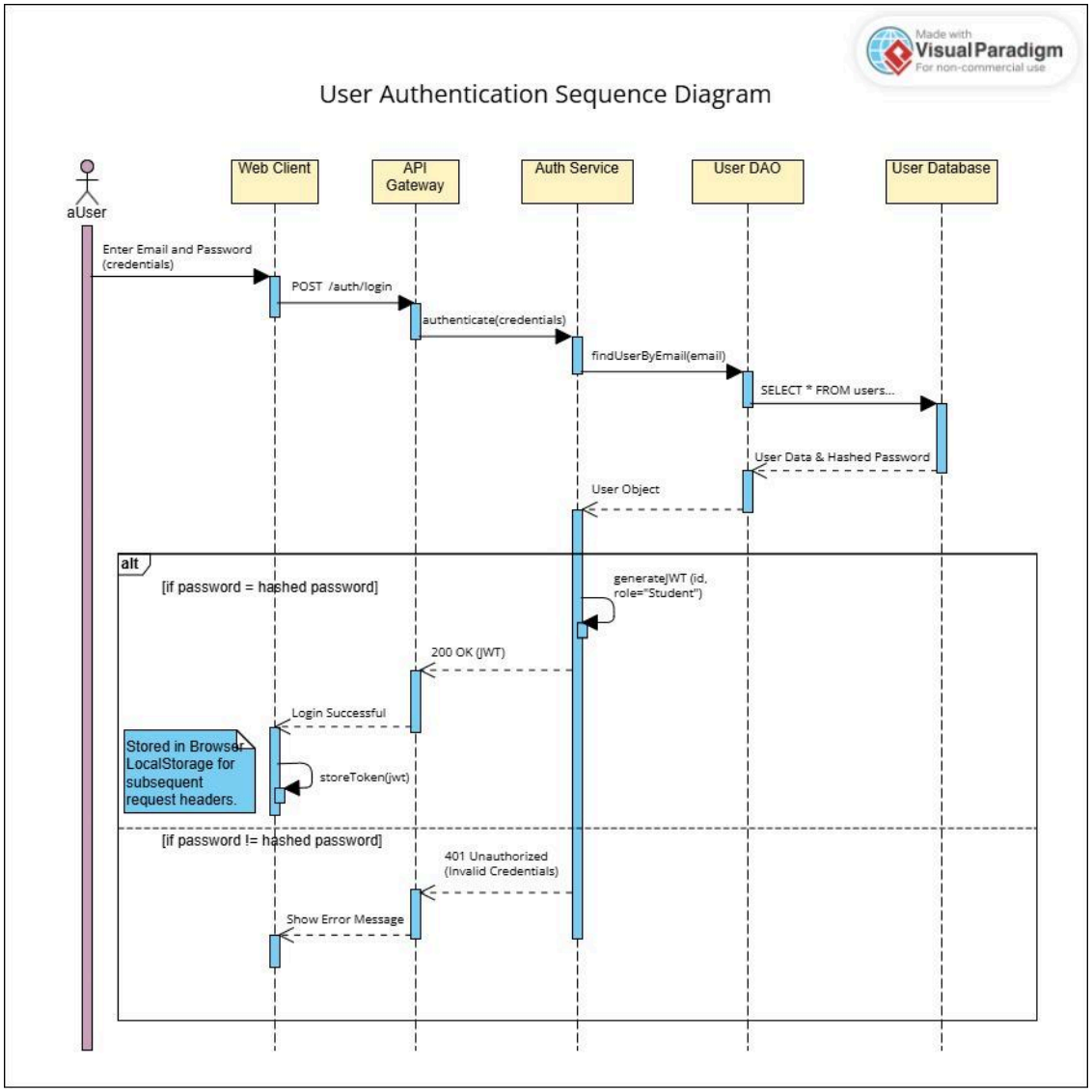
This system context diagram illustrates the boundaries of the Learning Management System, showing how it interacts with external actors like students and instructors while integrating with cloud infrastructure for hosting and data management. It provides a view of the entire ecosystem, detailing the data exchange between the core system and external entities to ensure seamless academic operations.

1.9.2 Component / Service Architecture Diagram

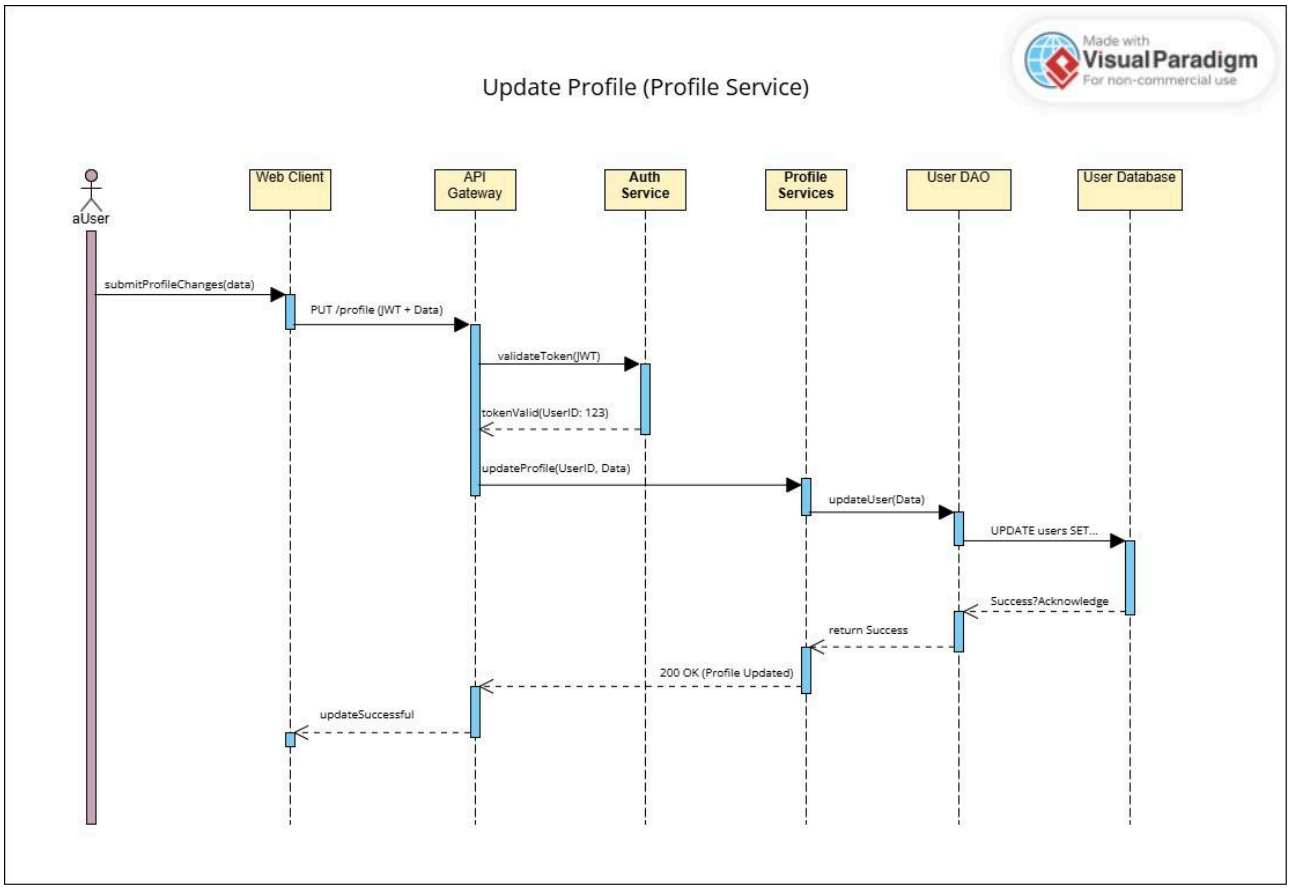


The component diagram for the Learning Management System illustrates a modular layered architecture, detailing the internal services like authentication and course management that power the platform's business logic. It provides a technical breakdown of how the API Gateway communicates between the web client and various data access objects to ensure organized data flow and system reliability.

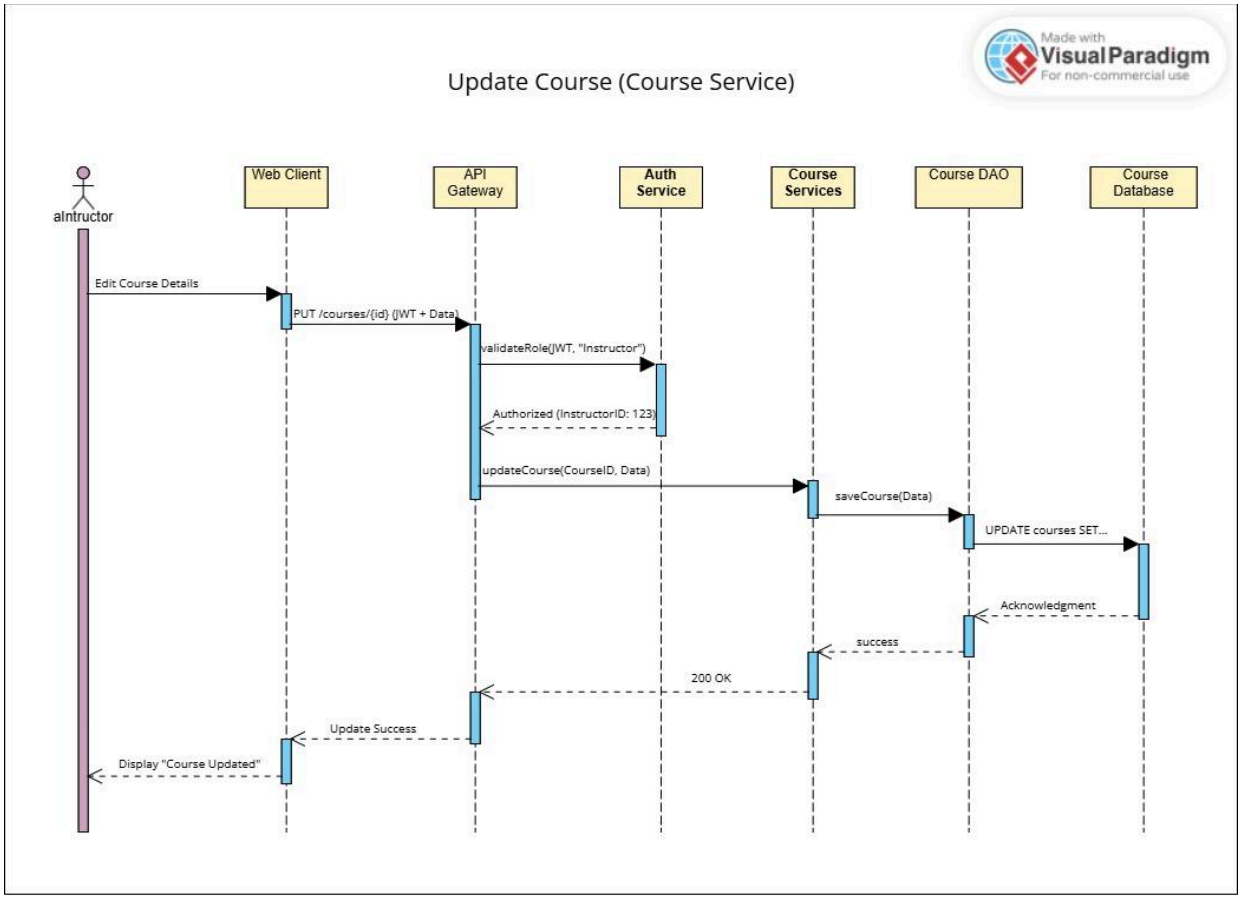
1.8.3 Interaction Diagram



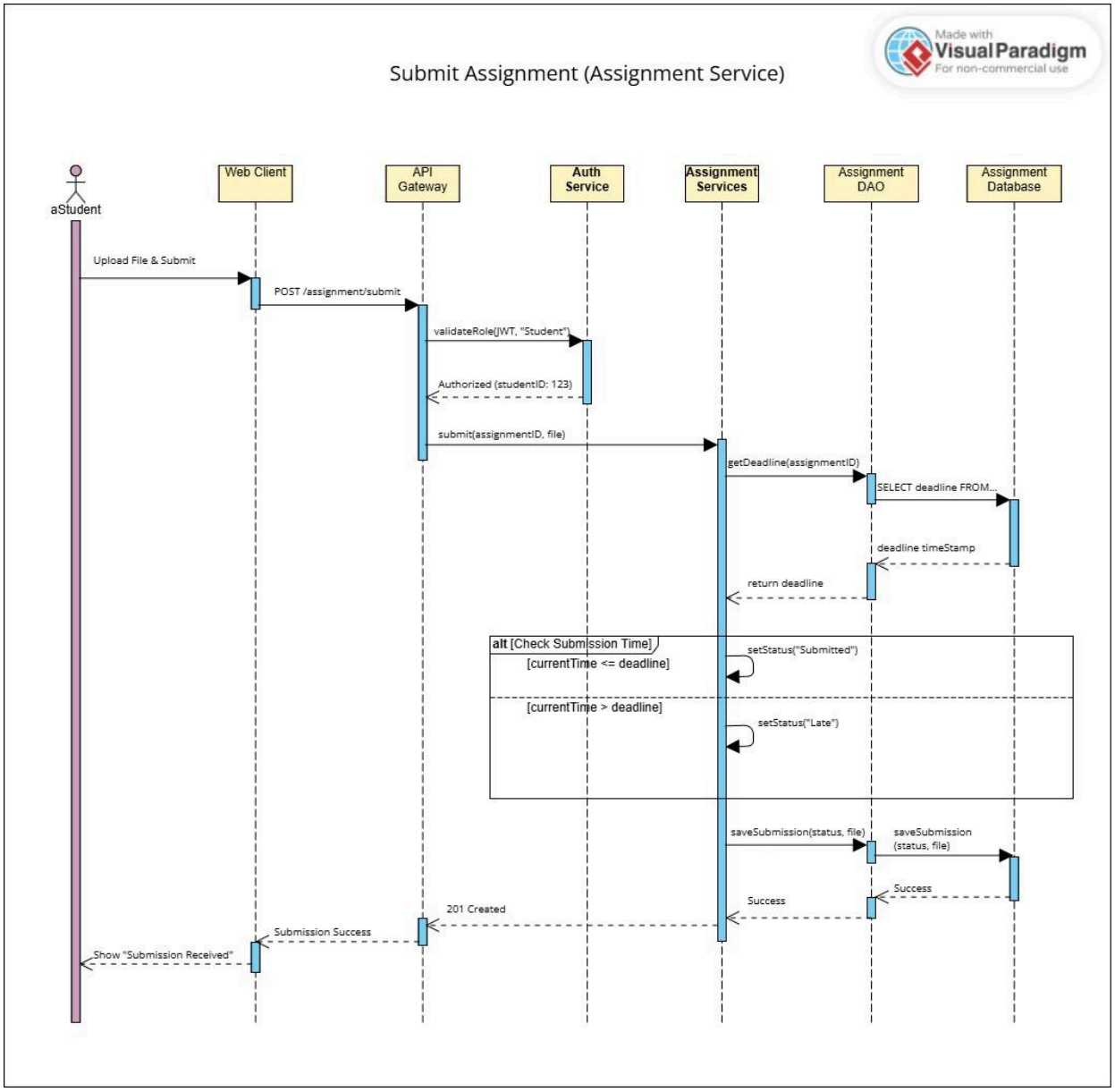
The User Authentication Sequence Diagram visualizes the step-by-step interaction between the user and the backend systems, outlining how credentials travel through the API Gateway and Auth Service for validation. It highlights the conditional logic used to issue a JWT upon successful verification or return an error message, ensuring a secure and responsive login flow.



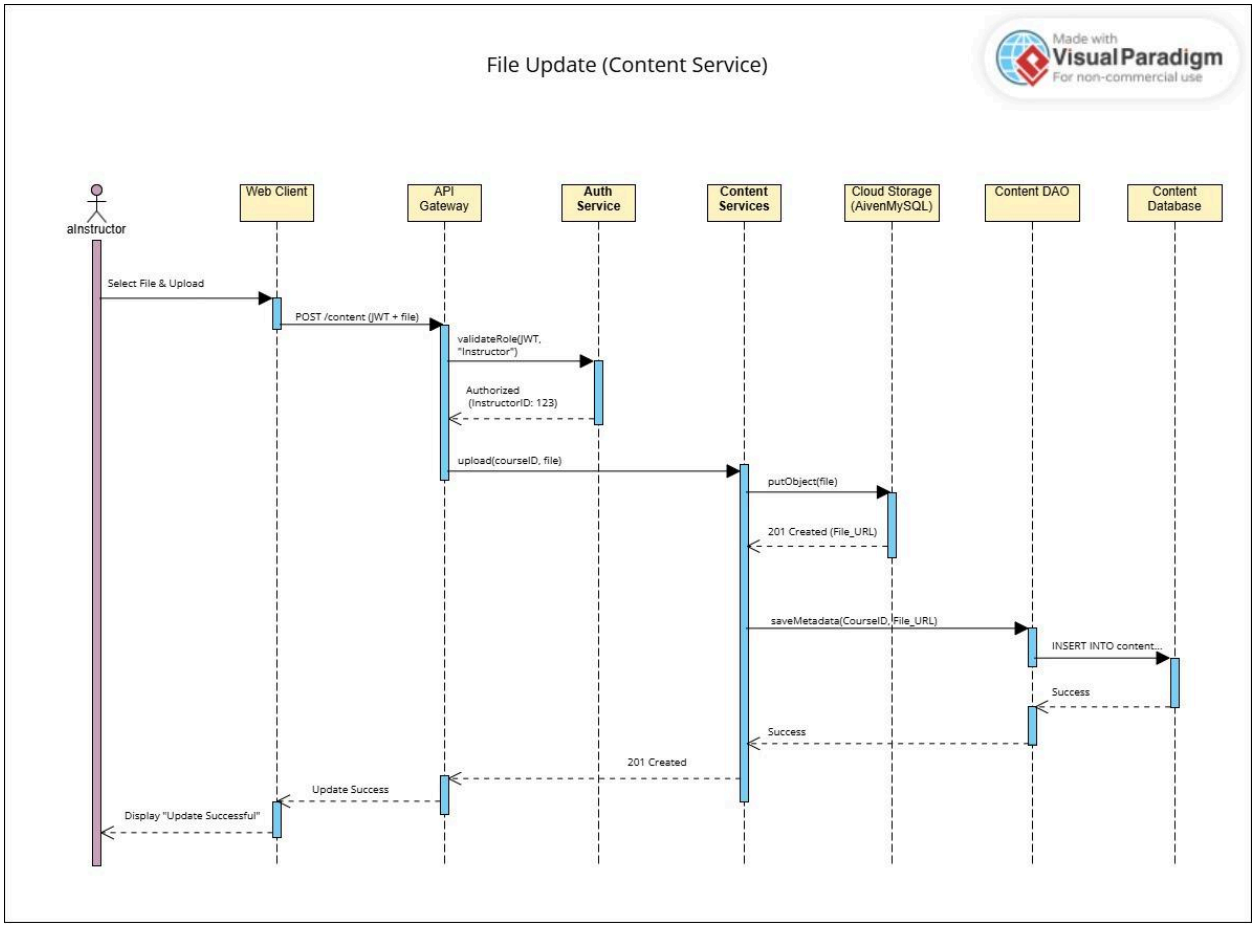
The Update Profile sequence diagram details the communication flow required to modify user information, starting with a secure token validation through the Auth Service. It demonstrates how the Profile Service interacts with the User DAO to commit changes to the database, ensuring that personal data updates are processed reliably and securely.



This interaction diagram depicts the specialized workflow for instructors to modify course details, specifically highlighting the role-based authorization check performed by the API Gateway. It follows the data path from the initial edit request through the Course Service and DAO, confirming that the course repository is updated only after the user's instructor status is verified.

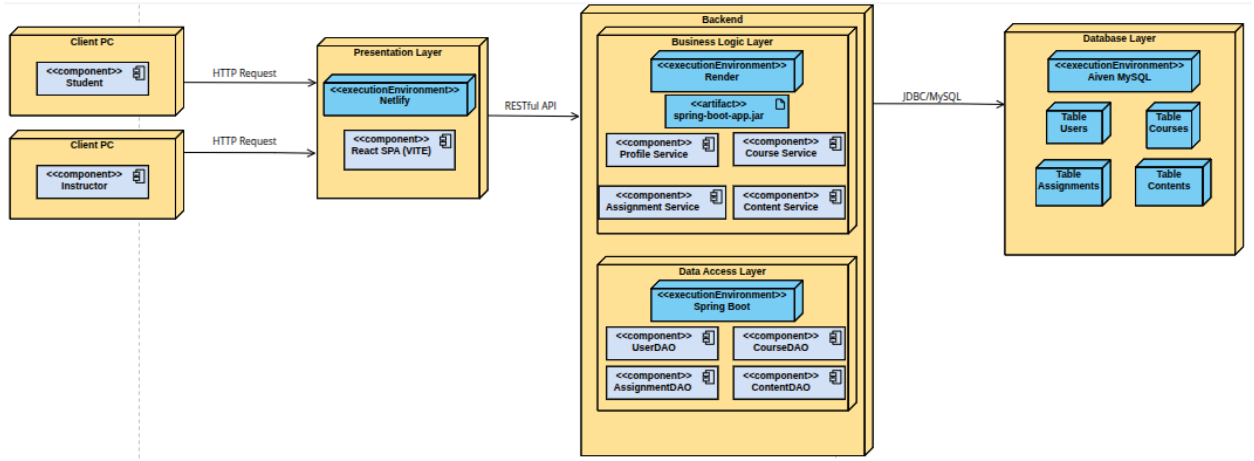


The Submit Assignment sequence diagram illustrates the workflow for students to upload files and the automated validation logic that checks the current time against the assignment deadline. It details how the Assignment Service interacts with the DAO to persist the file while also updating the submission status to either "Submitted" or "Late" based on the system's time-stamped check.



The File Update sequence diagram outlines the process of an instructor uploading new course materials, demonstrating a two-step storage strategy where the file is first pushed to cloud storage. It details how the Content Service captures the resulting file URL to save the metadata into the database via the Content DAO, ensuring that all uploaded resources are both securely hosted and correctly referenced within the Learning Management System.

1.9.4 Deployment Diagram



The deployment diagram provides a comprehensive map of the system's physical infrastructure, illustrating how the React-based frontend on Netlify communicates with the Spring Boot backend hosted on Render. It highlights the separation of concerns across the presentation, business logic, and data access layers, culminating in a secure JDBC connection to the Aiven MySQL cloud database for persistent storage.

1.10 Data ownership

Data	Owner (DAO)	Accessed By (Service)	Tables	Entities
User Accounts	UserDAO	Auth Service, Profile Service	Users	User credentials, roles, profile info
Course Data	CourseDAO	Course Service	Courses	Course details, schedules, enrollments
Assignment	AssignmentDAO	Assignment Service	Assignments	Assignments, submissions, grades, feedback, status
Content	ContentDAO	ContentDAO	Contents	File references, URLs, module organization

References

1. Cherif, Y. (2024, November 28). *Understanding the Layered Architecture Pattern: A Comprehensive guide*. DEV Community. https://dev.to/yasmine_ddec94f4d4/understanding-the-layered-architecture-pattern-a-comprehensive-guide-1e2j

2. *Home | Moodle.org*. (n.d.). https://moodle.org/

3. GeeksforGeeks. (2025, July 23). *Layered architecture of cloud*. GeeksforGeeks. https://www.geeksforgeeks.org/devops/layered-architecture-of-cloud/

4. Savolainen, J., & Myllarniemi, V. (2009, September). Layered architecture revisited—Comparison of research and practice. In *2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture* (pp. 317-320). IEEE.

5. Aldheleai, H. F., Bokhari, M. U., & Alammari, A. (2017). Overview of cloud-based learning management system. *International Journal of Computer Applications*, *162*(11), 41-46.

6. GeeksforGeeks. (2026, March 2). *REST API Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/node-js/rest-api-introduction/